

OOP & Software Testing – Add-on für IHK Algorithmen 2025 (FIAE)

Kurz, prüfungsnah, Java-ähnlicher Pseudocode (ohne Datentypen). Für die typischen AP2-Aufgaben.

1) OOP-Essentials (was die IHK gerne prüft)

Klasse & Objekt

```
klasse Artikel
  nummer
  preis

  konstruktor(nummer, preis)
    this.nummer = nummer
    this.preis  = preis

  funktion getNummer() return nummer
  funktion getPreis()  return preis
  funktion setPreis(p) preis = p
```

Merke: Attribute, Konstruktor, Getter/Setter – das ist der Standard.

Kapselung & Validierung

```
funktion setPreis(p)
  if (p < 0) p = 0 // einfache Validierung
  preis = p
```

Pluspunkte: Setter mit Prüflogik zeigen (keine negativen Preise).

Objektrückgabe

```
klasse Jahresstatistik(min, max)
```

```
funktion ermittleJahresstatistik(a)  
  // min/max berechnen ...  
  return new Jahresstatistik(min, max)
```

Typische Aufgabe: Objekt zurückgeben statt 2 Werte – zeigt OO-Denken.

Vergleichsfunktion (Comparator)

```
funktion sort(a, vergleiche)  
  for (i=0; i<a.length-1; i++)  
    for (j=0; j<a.length-1-i; j++)  
      if (vergleiche(a[j], a[j+1]) > 0)  
        swap(a[j], a[j+1])
```

Merke: Bei Objekten nie $x > y$ – immer `vergleiche(x,y)`.

Mini-Beispiele im IHK-Stil

- **„Teuerster Artikel“:** Durchlaufe Liste von Artikel, merke `maxPreis` und `maxArtikel`, gib das Objekt zurück.
 - **„Tageskurse sortieren“:** Übergib `vergleiche(x,y)`, das nach Datum vergleicht; nutze BubbleSort-Gerüst.
 - **„Jahresstatistik“:** Summiere pro Monat, ermittle min/max, gib `new Jahresstatistik(min,max)` zurück.
-

2) Software-Testing (knackig & prüfungsnah)

Whitebox-Test

- Quellcode ist bekannt (Pfad-/Zweigabdeckung).
- **Vorteile:** Fehlerursache präzise lokalisierbar, hohe Abdeckung messbar.
- **Beispiel:** If-Zweige gezielt mit Grenzwerten testen.

Blackbox-Test

- Nur Ein-/Ausgabe (Sicht des Anwenders/Spezifikation).
- **Vorteile:** Implementierungsunabhängig, realistische Nutzungsszenarien.
- **Beispiel:** notierungPlus mit typischen und Grenzfällen füttern.

Regressionstest

- Nach Änderungen prüfen, dass bestehende Funktionen noch korrekt sind.
- **Zweck:** Nebeneffekte/Fehlerrückkehr verhindern.
- **Beispiel:** Nach Refactoring laufen alte Testfälle unverändert „grün“.

Testfall-Techniken

- **Äquivalenzklassen:** Repräsentative Werte je Klasse.
- **Grenzwertanalyse:** Werte am Rand (z. B. -1, 0, 1).

Unit-Test Pseudocode (Java-ähnlich)

```
funktion test_notierungPlus()  
  // Arrange  
  kurse = [  
    {aktieProzent: 1.2, daxProzent: 0.8},  
    {aktieProzent: -0.5, daxProzent: 0.2},  
    {aktieProzent: 0.3, daxProzent: 0.1}  
  ]  
  erwartet = 2  
  
  // Act  
  ergebnis = notierungPlus(kurse)
```

```
// Assert
```

```
if (ergebnis != erwartet) fail("erwartet 2, bekam " + ergebnis)
```

Prüfungstricks:

- Bei „Vorteile Whitebox ggü. Blackbox“: *lokalisierbare Fehler, hohe Abdeckung.*
 - Bei „Regressionstest Zweck“: Nach Änderung sicherstellen, dass *Bestehendes* noch funktioniert.
 - Bei Testfällen: Nenne *Äquivalenzklassen* und *Grenzwerte*.
-

3) Mini-Zusammenfassung (für die letzte Minute)

- **OOP:** Klasse mit Attributen, Konstruktor, Getter/Setter, Objekt-Rückgabe, Comparator.
- **Testing:** Whitebox = Code-Sicht, Blackbox = I/O-Sicht, Regression = Alt-Funktionalität bleibt korrekt.
- **Form:** Java-ähnlicher Pseudocode, klar strukturiert, ohne Datentypen.

Druck-Tipp: Browser → Drucken → Als PDF speichern (weißes Layout).